TU Berlin, Fakultät IV
Brock/Pfisterer/Koenig
MAR 5-1

Aufgabenblatt 10

Effiziente Suche mit Hash Sets

Abgabe (bis 21.07.2025 23:59 Uhr)

Die folgenden Dateien werden für die Bepunktung berücksichtigt:

Geforderte Dateien:	
`Blatt10/src/Position.java`	Aufgabe 3.2
`Blatt10/src/Board.java`	Aufgabe 3.3
`Blatt10/src/PartialSolution.java`	sowie
`Blatt10/src/RicochetRobots.java`	Aufgabe 3.4

Als Abgabe wird jeweils nur die letzte Version im main branch in git gewertet.

Aufgabe 1: Hashing (Tutorium)

1.1
1. Wie funktioniert Hashing und was sind die Vorteile?
2. Was sind Kriterien für eine gute Hashfunktion?
3. Wo wird Hashing angewandt?

1.2 Sei k die Stelle eines Buchstaben im Alphabet. Betrachten Sie nun Hashing mit folgender Hashfunktion $h(k) = 11k \mod 7$. Fügen Sie damit nacheinander die Buchstaben des Worts ALGODAT in die Hashtabelle ein. Verwenden Sie dabei zur Kollisionsauflösung lineares Sondieren mit Inkrement 1 (d.h. $s(n) = n$).

0	1	2	3	4	5	6

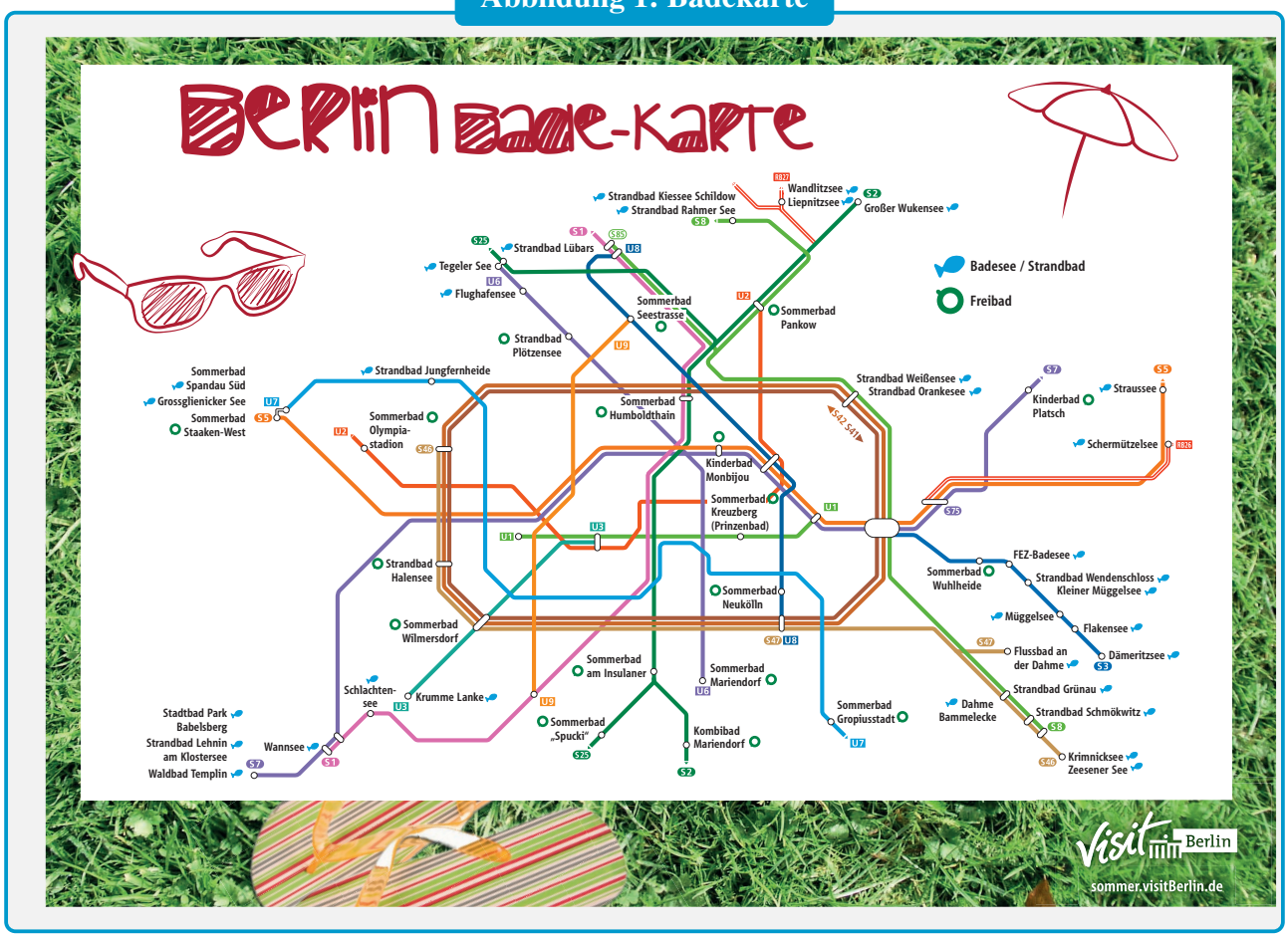
Aufgabe 2: Laufzeitanalyse (Tutorium)

2.1 Geben Sie möglichst genau die Wachstumsordnung für den folgenden Code an. Begründen Sie Ihre Herleitung der Laufzeit am Code. Gehen Sie davon aus, dass der Graph N Knoten enthält und E Kanten. `solved` ist hierbei ein `boolean` Array der Länge N.

```
public void mystery(Graph G, int v, boolean[] solved) {
    solved[v] = true;
    for (int w : G.adj(v)) {
        if (!solved[w]) {
            mystery(G, w, solved);
        }
    }
}
```

Was macht der Code? Kennen Sie diesen Algorithmus?

Abbildung 1: Badekarte



2.2 Erklären Sie an der Karte[1] in Abbildung 1, welche unterschiedlichen Konzepte von Graphen Sie kennengelernt haben, welche Algorithmen auf diesen Graphen Sie kennen und welche Arten von Problemen diese lösen.

[1] https://www.visitberlin.de/de/blog/berlin-bade-karte

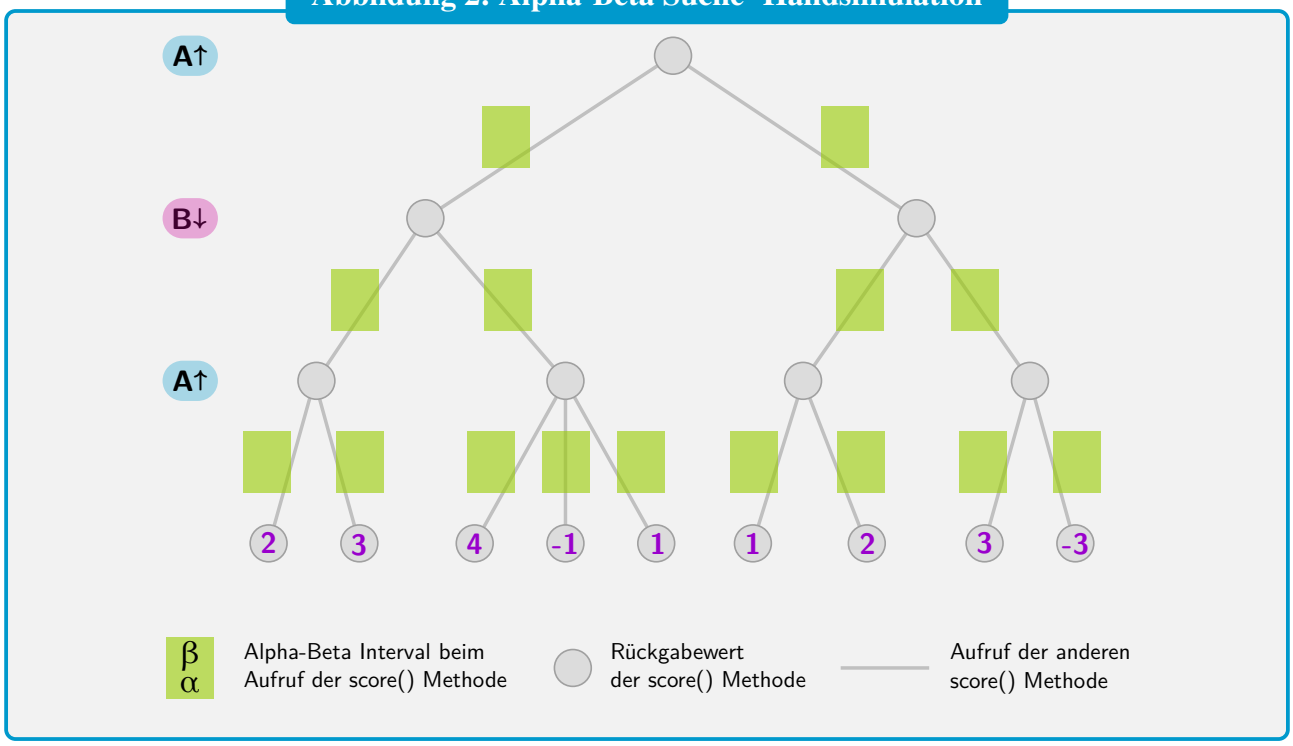
2.3 **(Optional)** Erstellen sie für folgende Kleidungsstücke Knoten und Kanten in einem gerichteten Graphen, wobei eine Kante bedeutet, dass der Startknoten vor dem Endknoten angezogen werden muss: Unterhemd (0), Socken (1), Krawatte (2), Gürtel (3), Schuhe (4), Hose (5), Sakko (6), Unterhose (7).

Geben sie anschließend eine Topologische Sortierung und eine Beispielreihenfolge für das Anziehen an, indem sie den folgenden Code per Hand simulieren. Wie muss dafür die Methode `public void mystery` angepasst werden? Für welche Graphen funktioniert der Algorithmus?

```
public void topologicalSortClothing(Graph g){
    Stack<Integer> stack = new Stack<Integer>();
    boolean visited[] = new boolean[G.V()];
    for (int i = 0; i < G.V(); i++){
        visited[i] = false;
    }
    for (int i = 0; i < V; i++){
        if (!visited[i]){
            mystery(G, i, visited, stack);
        }
    }
    while (!stack.isEmpty()){
        System.out.print(stack.pop() + " ");
    }
}
```

2.4 **(Optional)** Gehen Sie die Handsimulation der Alpha-Beta-Suche in Abbildung 2 schrittweise durch. Markieren Sie ggf. α- und β- Cutoffs.

Abbildung 2: Alpha-Beta Suche- Handsimulation

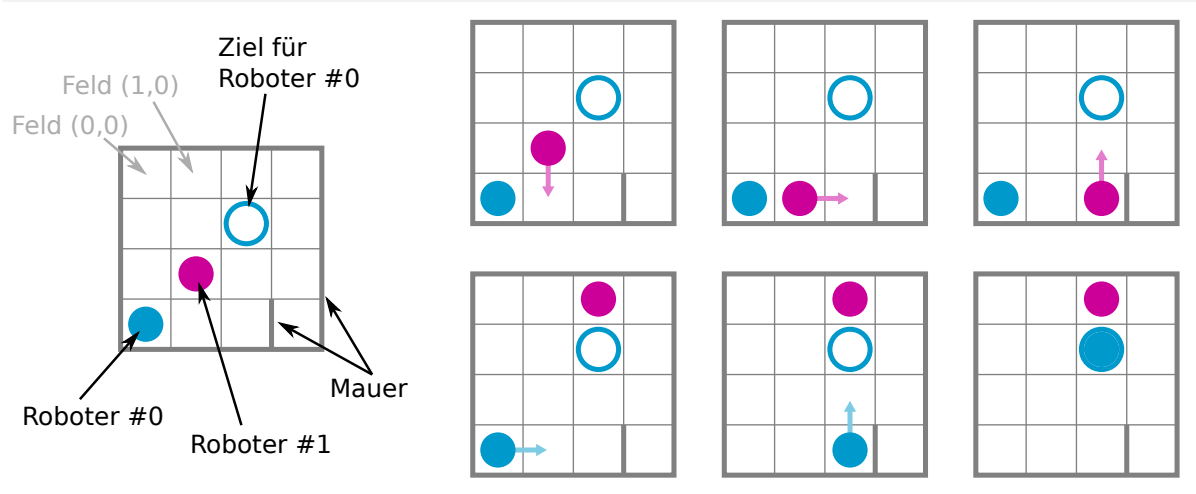


Aufgabe 3: Lösungsprozedur für ein Knobelspiel (Hausaufgabe)

Rasende Roboter[2] (*Ricochet Robots*) ist ein Brettspiel für mehrere Personen von Alex Randolph. Es kann auch alleine als Knobelspiel gespielt werden, auch auf anderen Brettern, als die Brettspielvariante. Es gibt viele Apps für Mobilgeräte, die dieses Spiel entweder direkt implementieren, oder davon inspiriert sind. Um neue Level für ein solches Spiel zu erzeugen, benötigt man einen sehr schnellen *Solver*. Für dieses Aufgabenblatt ist die Grundfunktionalität des Spieles gegeben. Alle spezifischen Bestandteile für eine effiziente Lösungsprozedur (und die Prozedur selber) sind zu implementieren.

Während das Original Brettspiel 16×16 Felder besitzt, ist bei uns die Größe des Spielfeldes variabel und wird meist deutlich kleiner sein. Ebenso variiert die Anzahl der Roboter. Das Spiel wird in Abbildung 3 erläutert.

Abbildung 3: Ricochet Robots



Das rechteckige Spielfeld besteht aus quadratischen Feldern, die durch Mauern getrennt sein können. Es gibt immer eine äußere Berandung durch Mauern. Bei einem Spielzug wird einer der Roboter in eine Richtung gezogen, und zwar soweit, bis er durch eine Mauer oder einen anderen Roboter gestoppt wird. Auch wenn dabei über mehrere Spielfelder hinweggezogen wird, zählt dies als *ein* Spielzug. Ein Bremsen auf freier Strecke ist nicht möglich.
Ein Feld ist als Zielfeld für einen der Roboter markiert. Ziel des Spieles ist es, eine möglichst kurze Zugfolge zu finden, durch die dieser Roboter auf dem Zielfeld zum Stehen kommt. Ein Roboter rutscht allerdings über das Zielfeld hinweg, falls er nicht durch eine Mauer oder einen anderen Roboter gestoppt wird. (Beispieldatei: `rrBoard-sample01.txt`)

In dieser Aufgabe soll eine effiziente Lösungsprozedur für Ausgangssituationen von Ricochet Robots mit der Hilfe von *Hash Sets* implementiert werden. Anders als bei dem 15-Puzzle des letzten Aufgabenblattes ist bei Ricochet Robots nicht klar, wie eine konsistente Heuristik formuliert werden könnte. Es gibt nicht einmal einen Ansatz für eine (sinnvolle) zulässige Heuristik, die als Schranke für ein *Branch-and-Bound* Verfahren dienen könnte. Somit gibt es zunächst nur den Ansatz der Breitensuche. Bei diesem Spiel ergibt sich dabei das Problem, dass man durch viele unterschiedliche Spielfolgen zu derselben Spielsituation gelangt. Wenn dies nicht berücksichtigt wird, wächst der Suchbaum der

[2] https://de.wikipedia.org/wiki/Rasende_Roboter

Breitensuche (und in der Implementation konkret die Warteschlange) so schnell an, dass schon relativ einfache Probleme nicht in akzeptabler Rechenzeit gelöst werden können.

Daher prüft man nach der Ausführung eines Zuges, ob die neue Spielsituation schon vorher erreicht wurde und verwirft sie in diesem Fall. Um den Vergleich einer Spielsituation mit allen früheren Spielsituationen effizient zu gestalten, werden *Hash Sets* für die Klasse `Board` verwendet.

3.1 Methoden für Spiellogik studieren (0 Punkte)

Betrachten Sie die gegebenen Klassen `Position`, `Move`, `Robot` und `Board`, die die Spiellogik von Ricochet Robots implementieren. Die `main()` von `Board` enthält Beispielcode, der direkt ausgeführt werden kann.

Die variablen Elemente bei einem einzelnen Spiel sind nur die Positionen der Roboter. Daher wird bei diesem Spiel die Spielsituation nicht über ein zwei-dimensionales Array mit den Inhalten jedes Spielfeldes gespeichert, sondern als eine Liste der Positionen der Roboter.

Um einen Hashcode für die Klasse `Board` zu implementieren, benötigt man also einen Hashcode für die Klasse `Position`. Fangen wird damit an:

3.2 Klasse `Position` vervollständigen (10 Punkte)

Implementieren Sie die Methoden

```
@Override
public boolean equals(Object o)
@Override
public int hashCode()
```

in der Klasse `Position`. Die Hashcodes von `Position` Objekten sollten für alle Positionen auf einem 20×20 Brett unterschiedlich sein.

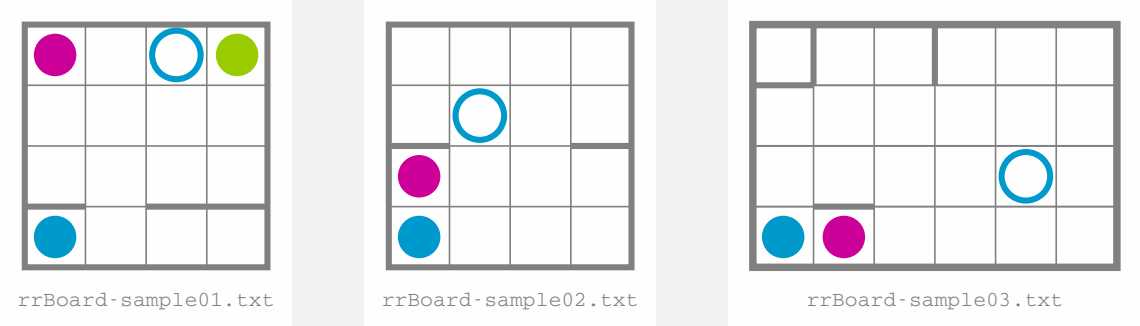
3.3 Klasse `Board` vervollständigen (20 Punkte)

Implementieren Sie die Methoden

```
@Override
public boolean equals(Object o)
@Override
public int hashCode()
```

in der Klasse `Board`. Sie brauchen in beiden Methoden nur diejenige Information zu berücksichtigen, die sich während eines Spielverlaufes (bzw. während der Lösung einer Ausgangsstellung) ändert.

Abbildung 4: Drei Knobelaufgaben



Hier sind drei weitere *Ricochet Robots* Aufgaben. Ziel ist es, den blauen Roboter in das blaue Zielfeld zu bewegen. Die Lösungen für die ersten beiden sind weiter unten abgebildet. Die dritte bleibt zum Knobeln. Oder warten Sie auf Ihre Implementierung des Solvers!
Alle drei Aufgaben sind in den Vorgaben als Dateien im `samples` Verzeichnis gegeben. Zu den ersten beiden (sowie dem Spiel aus Abb. 3) sind die Lösungen des Solvers ebenfalls mit dabei. Die `main()` Methode der Klasse `RicochetRobots` enthält Code zum Laden der Beispiele, Aufruf des Solvers und Ausgeben der Lösung.

3.4 Implementation der Klasse `PartialSolution` und `RicochetRobots` (70 Punkte)

Implementieren Sie in der Klasse `PartialSolution` die notwendige Funktionalität für die Lösungssuche. Die Implementierung ist eng verknüpft mit der Implementierung der Lösungssuche in der Klasse `RicochetRobots` (siehe nächster Aufgabenteil). Sie sollten also das Konzept für beide Klassen gemeinsam planen.

Hier ist die Aufgabe freier gestaltet, als auf dem letztem Aufgabenblatt. Sie können also selbst entscheiden, wieviel Funktionalität Sie in die Klassen `PartialSolution` verlegen möchten. Im Minimum muss `ParitialSolution` eine Spielstellung `Board` und die Zugsequenz dorthin speichern. Andere Funktionalität wie *isFinal* oder *doMove* kann direkt über `Board` in der Methode für die Breitensuche abgerufen werden oder wie auf dem letzten Aufgabenblatt in `PartialSolution` integriert werden.

Die einzige Vorgabe dabei ist, dass die Methode

```
public LinkedList<Move> moveSequence()
```

implementiert sein muss. Der Rückgabewert ist eine Liste. Es wird nicht überprüft, was in der Liste steht, sondern nur, ob die Länge der Liste stimmt, also die minimale Anzahl der Spielzüge.

Implementieren Sie in der Klasse `RicochetRobots` die Methode

```
public static PartialSolution bfsWithHashing(Board board)
```

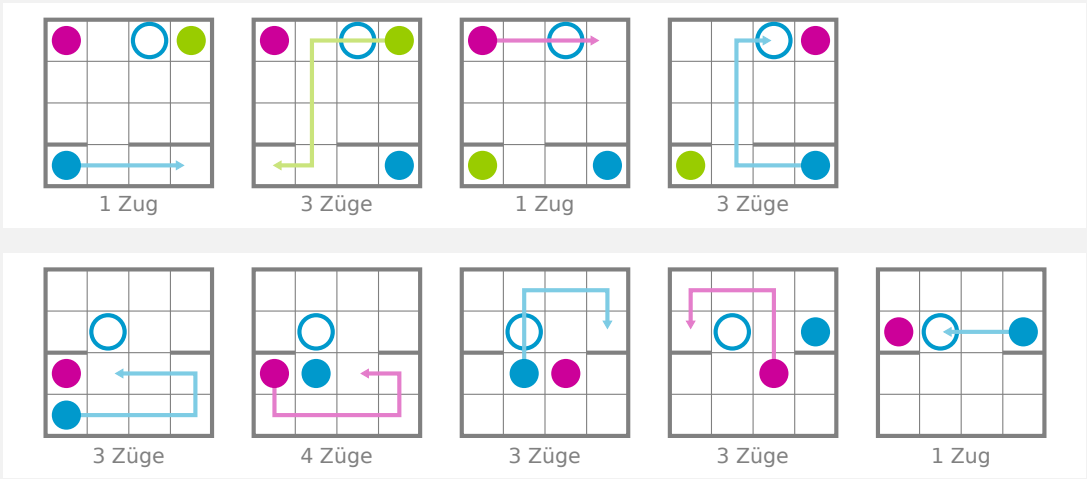
um eine kürzeste Lösung (geringste Anzahl von Spielzügen) einer beliebigen Spielstellung (Brettgrößen bis 20×20 und bis zu 10 Robotern) zu bestimmen. Um die geforderte Effizienz zu erreichen, soll bei der Lösungssuche mittels eines *Hash Sets* verhindert werden, dass dieselbe Spielsituation mehrfach untersucht wird. Ohne diese Optimierung explodiert der Lösungsraum und es wird fast kein Test ohne *timeout* bestanden.

Es ist sinnvoll zu testen, ob das Aussondern doppelter Spielstellungen mittels Hashing greift.

Hinweis:

- Die größere Freiheit bei der Wahl der Implementierung bringt es mit sich, dass keine einzelnen Aspekte in unseren Tests überprüft werden können. Sie beruhen also im Wesentlichen auf einer Überprüfung der Gesamtfunktionalität. Es gibt also keine speziellen Tests für `PartialSolution`.
- Eine Implementierung von `bfsWithHashing()` kommt mit 20 Zeilen (im normalen Programmierstandard) aus, selbst wenn `PartialSolution` nur die Minimalfunktionalität zur Verfügung stellt. Dies ist keine Aufforderung dazu, eine möglichst kurze Methode zu schreiben, sondern soll nur als Orientierung dienen. Falls Sie eine Idee haben, die sehr viel mehr Code und Hilfsfunktionen erfordert, schauen Sie nochmal in die Vorlesung und überlegen Sie, wie Sie die Aufgabe anders lösen können.
- Es sollte möglich sein, mehrere Runden Ricochet Robots hintereinander auf neuen Boards zu spielen.

Abbildung 5: Lösungen der Aufgaben aus Abb. 4



Was Sie nach diesem Blatt wissen sollten:

- Was Hashing ist
- Was perfektes Hashing ist und was sind Kollisionen sind
- Was Strategien zur Kollisionsauflösung sind und wie diese genau funktionieren
- Was die Besonderheiten beim Einfügen und Löschen von Schlüsseln sind und was lazy und eager deletion sind
- Was der Belegungsfaktor ist und welchen Einfluss er auf die Laufzeit hat
- Wie Hashing mit Java-Objekten funktioniert